



Curso: Desenvolvimento de Software Multiplataforma

Disciplina: Programação para Dispositivos Móveis I

Professor: André Olímpio

Programação Para Dispositivos Móveis I



Acesso a Recursos do Dispositivo

- Enviar mensagens SMS, WhatsApp e e-mail.
- Acessar a lista de contatos do dispositivo.
- Acessar a câmera do dispositivo.



Fonte: GizModo.UOL.com.br

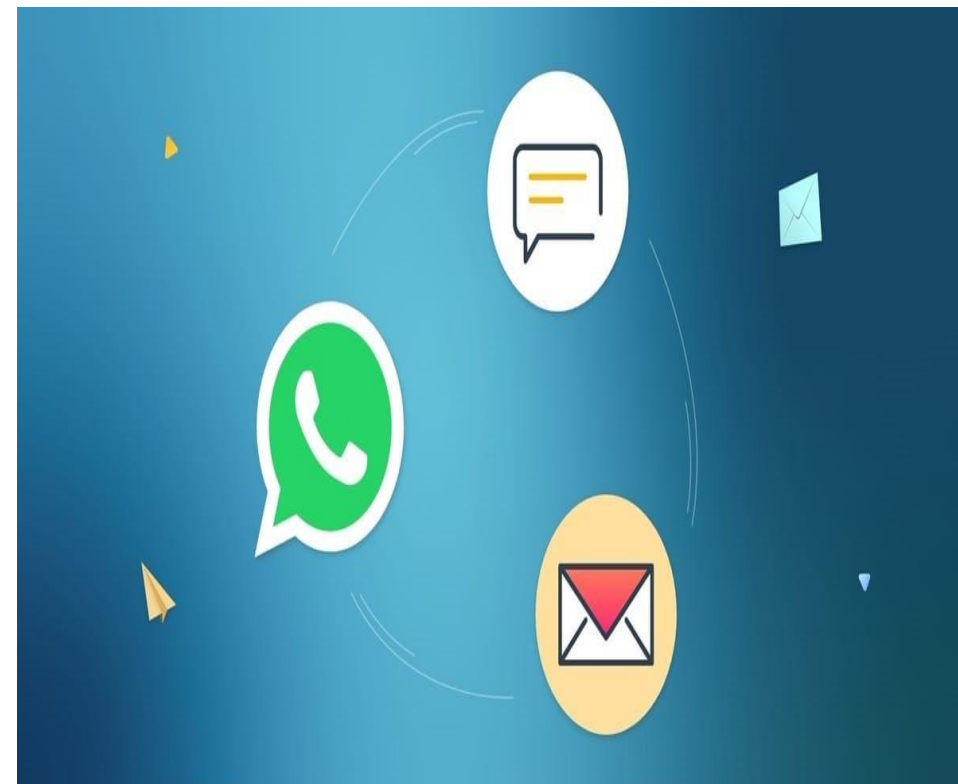
Enviar mensagens SMS, WhatsApp e E-mail

- No React Native, o módulo **Linking** é utilizado para interagir com aplicativos e funcionalidades externas ao nosso aplicativo, como abrir URLs, iniciar chamadas telefônicas, enviar mensagens de texto ou e-mails, abrir aplicativos de redes sociais.
- Fornece uma interface para acessar e manipular links universais ou profundos (deep links).

Deep links: são URLs que levam o usuário diretamente para um conteúdo específico dentro de um aplicativo.

Enviar mensagens SMS, WhatsApp e E-mail

- O uso de Linking permite que o aplicativo inicie o envio de uma mensagem SMS abrindo o serviço de mensagens com o número e o conteúdo da mensagem já preenchidos, mas o envio final ainda precisa ser confirmado pelo usuário.
- Ou seja, o código apenas dispara a abertura do aplicativo de mensagens do dispositivo.



Fonte: Blog.PepperCloud.com

Linking

- No React Native Expo, o conceito de **Linking** refere-se à capacidade de interagir com URLs e realizar navegação entre diferentes recursos dentro ou fora do aplicativo.
- Permite que o aplicativo se comunique com outros aplicativos ou abra páginas da web diretamente, sem a necessidade de sair do ambiente do app.



Fonte: Blog.PepperCloud.com

Linking

- O **Linking** é uma API que facilita a manipulação de links para:
 - 1. Abrir URLs externas:** É possível abrir uma URL em um navegador ou em outro aplicativo. Por exemplo, abrir uma página da web ou um link para o WhatsApp, e-mail.
 - 2. Navegação dentro do app:** É possível permitir que o app responda a URLs específicas e realize navegação interna, similar a como a navegação por rotas funciona em um navegador.

Linking

- Nem todos os dispositivos podem suportar o esquema **SMS**.
- Por isso, é necessário utilizar o método **canOpenURL** para verificar se o dispositivo pode processar a URL antes de tentar abri-la.
- O Linking abre outro aplicativo para enviar a mensagem, não requerendo permissões especiais como **SEND_SMS** (<https://reactnative.dev/docs/permissionsandroid>), que seria necessária para enviar SMS sem a intervenção do usuário.

Linking

- O *Linking* cria uma URL que segue o esquema **SMS**, que o sistema operacional reconhece e abre o aplicativo de mensagens padrão.

``sms:1234567890?body=${encodeURIComponent(message)}``

- **Formato da URL:**

- **sms:** é o esquema de URL que indica ao sistema que o aplicativo de mensagens deve ser aberto.
- **1234567890:** é o número de telefone do destinatário.
- **?body:** é o parâmetro que recebe o corpo da mensagem.
- **encodeURIComponent(message):** garante que a mensagem seja codificada corretamente para ser passada como parâmetro na URL.

Linking

O Linking **NÃO** envia a mensagem automaticamente, apenas abre o aplicativo de mensagens com a mensagem preenchida. **O usuário precisa confirmar o envio.**

- Para enviar mensagens pelo WhatsApp, é possível utilizar o Linking de maneira semelhante ao envio de SMS.
- A URL segue o formato `https://wa.me/`, onde `wa.me` é um atalho oficial do WhatsApp para iniciar conversas.
- O parâmetro `text` permite incluir uma mensagem pré-preenchida.

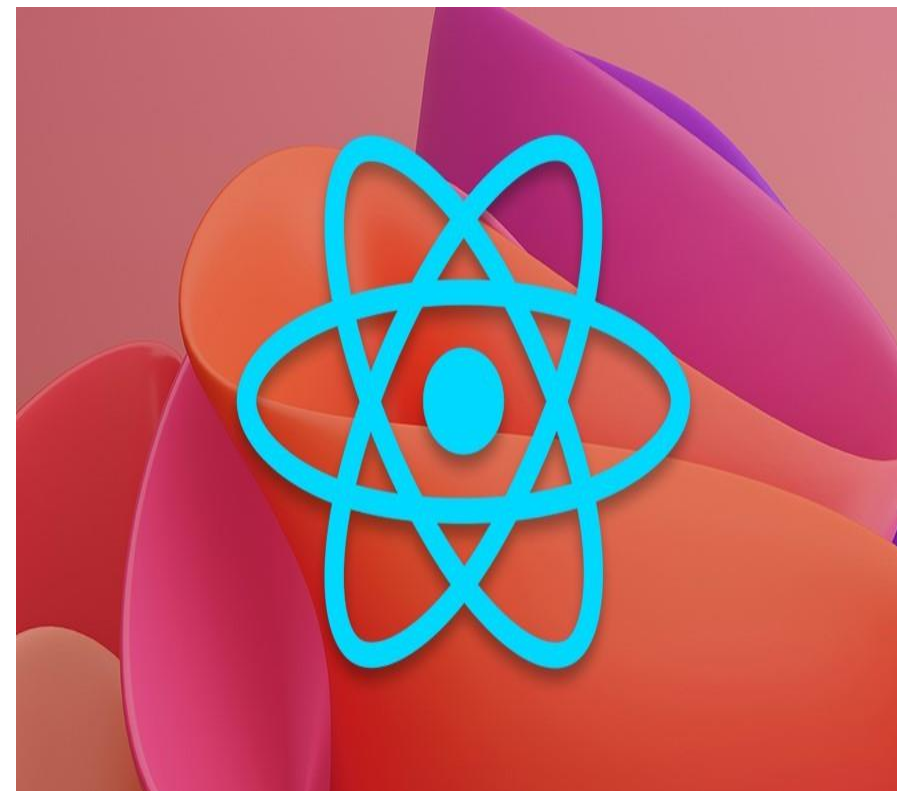
```
const url = `https://wa.me/${phoneNumber}?text=${encodeURIComponent(message)}`;
```

Acessar a Lista de Contatos do Dispositivo

- O pacote ***expo-contacts*** (<https://www.npmjs.com/package/expo-contacts>) é uma biblioteca fornecida pelo Expo que permite acessar os contatos armazenados no dispositivo do usuário em um aplicativo RN.
- **Principais funcionalidades:**
 - **Solicitar permissões:** antes de acessar os contatos, é necessário solicitar permissão ao usuário. O pacote fornece métodos para facilitar essa solicitação.
 - **Acesso aos contatos:** o expo-contacts permite acessar os contatos armazenados no dispositivo, podendo obter detalhes como nome, número de telefone, e-mail, endereço.
 - **Filtragem e busca:** permite buscar contatos específicos ou filtrar a lista de contatos com base em critérios como nome, número de telefone, e-mail.

Acessar a Lista de Contatos do Dispositivo

- Primeiramente é necessário instalar o pacote:
 - `npm i expo-contacts`
- O expo-contacts é compatível com ambas as plataformas (Android e iOS), embora existam algumas diferenças de implementação entre elas.



Fonte: [Blog.LogRocket.com](https://blog.logrocket.com)

Acessar a Lista de Contatos do Dispositivo

- Operações disponíveis:
- Solicitar permissão para acessar os contatos do dispositivo;

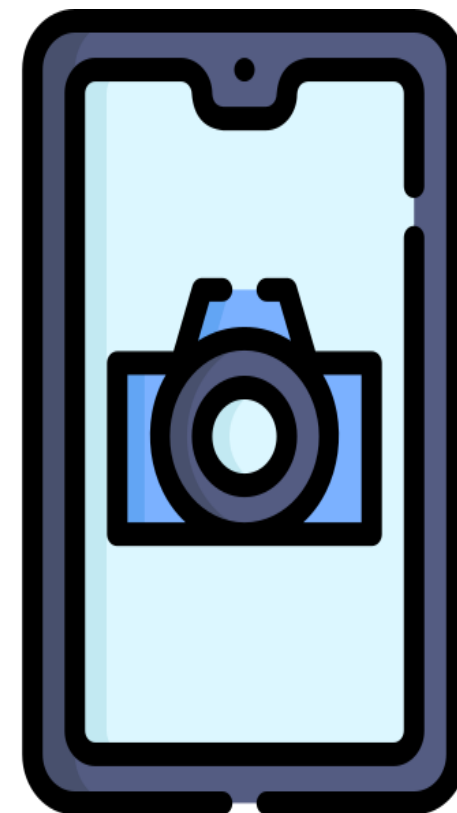
```
await Contacts.requestPermissionsAsync();
```

Se o resultado do pedido de concessão for *granted* (concedido), então é possível acessar a lista de contatos utilizando o método ***getContactsAsync***.

- Ler a lista de contatos armazenados no dispositivo.
- Filtrar contatos por campos específicos, como número de telefone ou e-mail.
- Adicionar, atualizar ou excluir contatos - em alguns casos, dependendo do suporte da plataforma.

Acessar a Câmera do Dispositivo

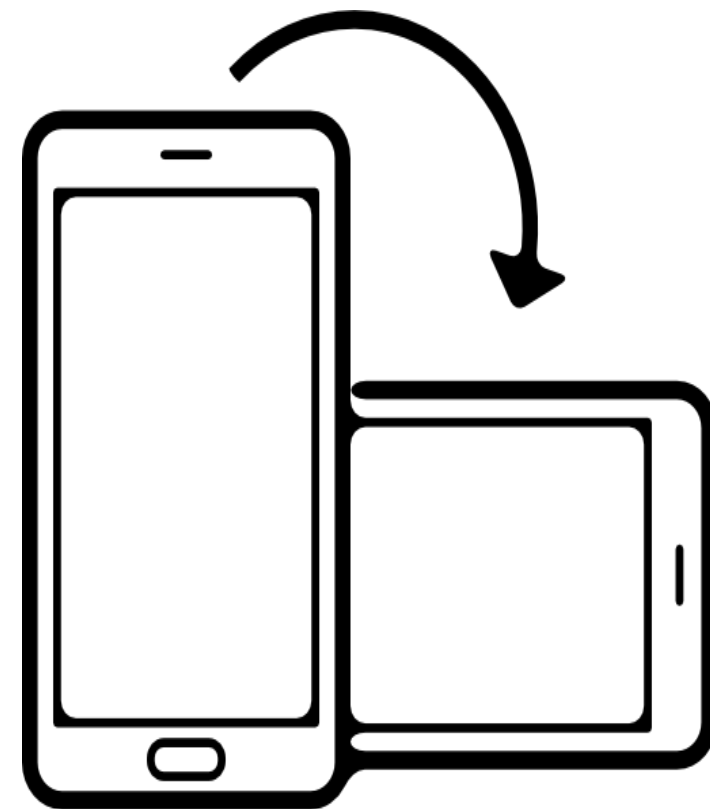
- O pacote ***expo-image-picker*** fornece acesso à interface do usuário do sistema para selecionar imagens e vídeos da biblioteca do dispositivo.
- Também permite tirar fotos ou gravar vídeos com a câmera integrada no dispositivo (<https://docs.expo.dev/versions/latest/sdk/imagepicker>).



Fonte: Flaticon.com

Orientação de Tela

- É definida como a orientação em que os gráficos são exibidos no dispositivo.
- Quando o dispositivo está na vertical a tela está na orientação **portrait** e, caso contrário, estará em **landscape**.
- <https://docs.expo.dev/versions/latest/sdk/screen-orientation>



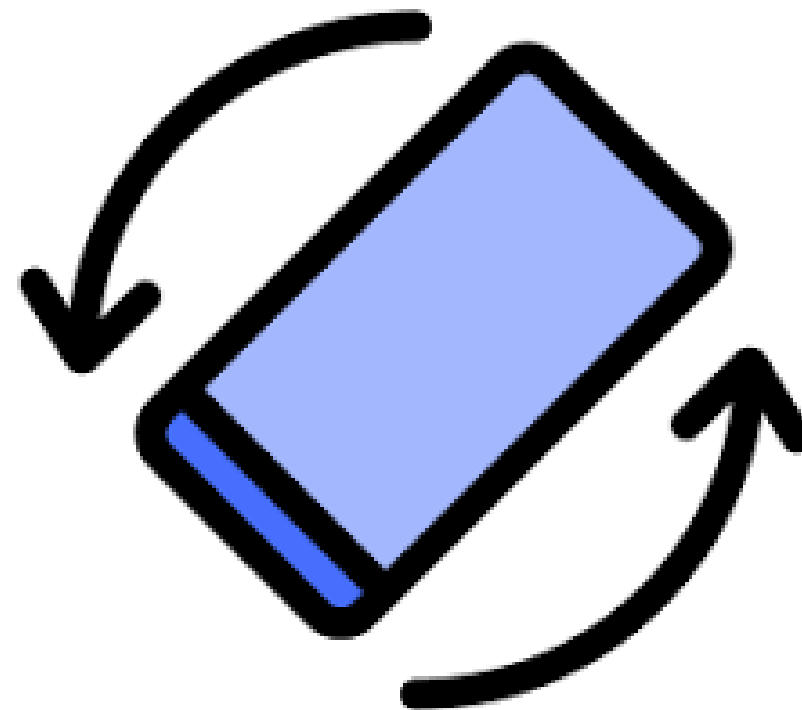
Fonte: Medium.com

Orientação de Tela

- Para fazer a gestão da orientação da tela em landscape ou portrait no RN com Expo, é possível utilizar o módulo *ScreenOrientation* da biblioteca *expo-screen-orientation*.
- <https://www.npmjs.com/package/expo-screen-orientation>
- Para reproduzir os exemplos crie um projeto RN com Expo e adicione a biblioteca *expo-screen-orientation*:
 - **npm i expo-screen-orientation**

Orientação de Tela

- A propriedade `orientation`, no arquivo `app.json`, da configuração do Expo define a orientação da tela que será utilizada durante a execução do aplicativo, que controla se o aplicativo será exibido apenas em modo “portrait” (vertical), “landscape” (horizontal) ou se permitirá ambos os modos “default”.



Fonte: FreePik.com

Orientação de Tela

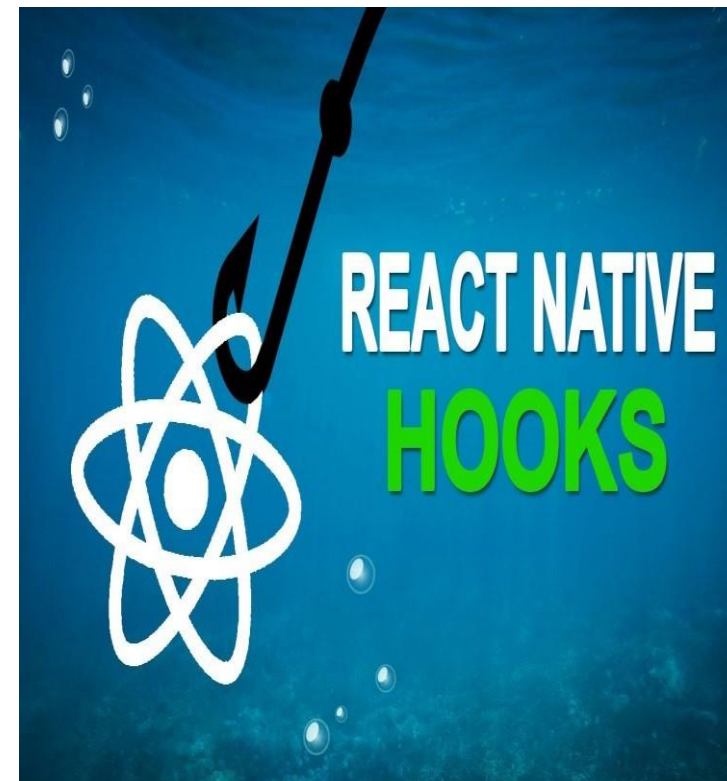
- **getOrientationAsync():** obtém a orientação atual da tela de forma assíncrona.
- **addOrientationChangeListener:** adiciona um listener que é chamado sempre que a orientação da tela muda.
- **ScreenOrientation.removeOrientationChangeListener(subscription):** remove o listener de mudança de orientação quando o componente for desmontado, evitando possíveis vazamentos de memória.

Orientação da Tela

- **Listener:** é um conceito fundamental na programação, especialmente em aplicações interativas. É um mecanismo que permite que um código reaja a determinados eventos que ocorrem dentro de um sistema, como cliques de mouse, pressionamento de teclas, envio de formulários, entre outros.
- Na prática, um Listener é uma função que “escuta” um evento específico e executa uma ação quando esse evento acontece.
- Em linguagens como JavaScript, os Listeners são amplamente utilizados para criar interações dinâmicas em páginas web.

Hooks

- É uma função especial que permite “ligar” o código do seu componente a funcionalidades do React, como o controle de estados e efeitos colaterais.
- Esses hooks ajudam a deixar o componente mais dinâmico, reagindo a mudanças de dados ou a momentos específicos do ciclo de vida da tela.



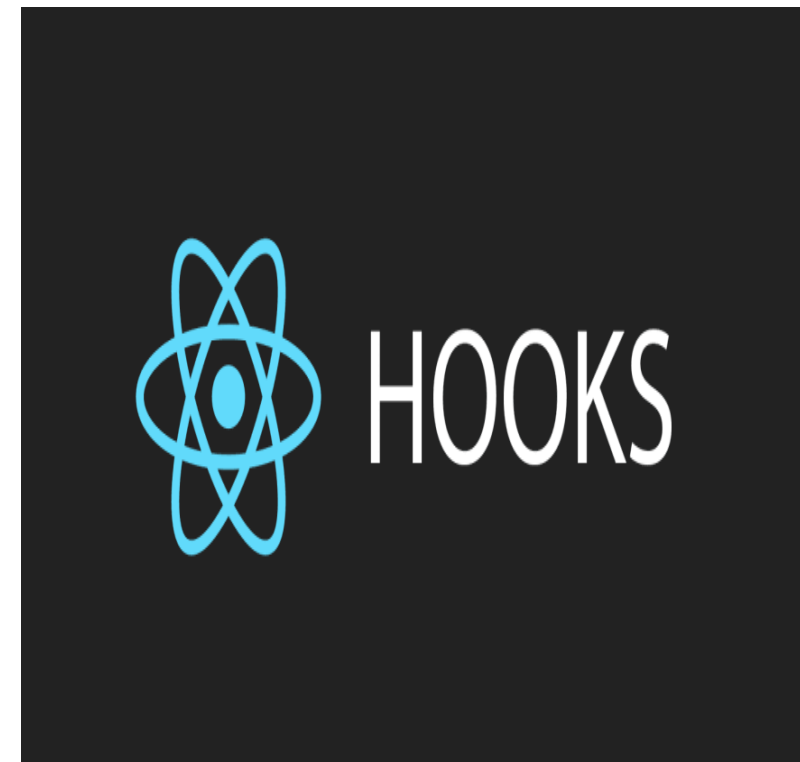
Fonte: Medium.com

Hooks

- No contexto do desenvolvimento mobile com **React Native**, os **Hooks** representam um avanço significativo na forma trabalhar com estado, ciclo de vida e lógica compartilhada em componentes.
- Introduzidos a partir da versão 16.8 do React, permitem que funcionalidades antes restritas a componentes de classe sejam utilizadas diretamente em **componentes funcionais**, promovendo um código mais limpo, conciso e de fácil manutenção.

Hooks

- O principal objetivo é simplificar a criação de componentes reutilizáveis e funcionais, eliminando a necessidade de utilizar classes para acessar recursos como o **estado local (state)** ou **métodos do ciclo de vida**.
- Isso melhora a legibilidade e a organização do código, especialmente em aplicações com lógica complexa ou com vários estados interdependentes.



Fonte: Medium.com

Principais Hooks utilizados em React Native

- **useState**
 - Esse Hook permite declarar e gerenciar valores de estado dentro de um componente funcional.
 - É frequentemente utilizado para armazenar valores como dados de entrada do usuário, contadores, indicadores booleanos e qualquer outra informação que precise ser reativa dentro da interface.

Principais Hooks utilizados em React Native

- **Exemplo:** contador de cliques simples do mouse

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

const Contador = () => {
  const [contador, setContador] = useState(0);

  return (
    <View>
      <Text>Você clicou {contador} vezes</Text>
      <Button title="Clique aqui" onPress={() => setContador(contador + 1)} />
    </View>
  );
};

export default Contador;
```

Fonte: Elaborado pelo autor

Principais Hooks utilizados em React Native

- **useEffect:**
 - Utilizado para administrar e controlar efeitos colaterais, executando uma função sempre que o componente é renderizado, montado, desmontado ou quando determinadas variáveis mudam.
 - É ideal para chamadas de API, timers, subscrições e atualizações de DOM (ou elementos nativos no caso do React Native), integrando comportamentos dinâmicos à interface.

Principais Hooks utilizados em React Native

- **useRef:**
 - Esse Hook permite criar referências mutáveis que persistem durante todo o ciclo de vida do componente, sem disparar re-renderizações.
 - No RN, é comum seu uso para manipular diretamente elementos da interface, como campos de entrada, animações ou armazenar valores temporários.

Principais Hooks utilizados em React Native

- **useContext:**

- É utilizado para consumir valores fornecidos por um contexto criado previamente com *createContext*.
- Facilita o compartilhamento de dados globais, como temas, preferências do usuário ou dados de autenticação, evitando o repasse manual de props por vários níveis da árvore de componentes.

Benefícios do Uso de Hooks em Aplicativos Móveis

- Redução significativa da complexidade de componentes.
- Eliminação da necessidade de classes para gerenciar estado ou ciclo de vida.
- Maior facilidade para dividir e reutilizar lógica entre componentes diferentes.
- Aumento da coesão e da legibilidade do código.

Os Hooks proporcionam uma transição mais natural para conceitos funcionais e tornam o código mais compatível com boas práticas modernas de desenvolvimento.

Custom Hooks

- Além dos Hooks fornecidos pela biblioteca do React, é possível criar Hooks personalizados.
- Esses são utilizados para encapsular e reutilizar lógica específica, como chamadas a serviços externos, validações ou manipulação de estado complexo.
- Ao fazer isso, é possível manter os componentes mais simples e modulares.

Leitura Complementar

- Expo SMS

<https://docs.expo.dev/versions/latest/sdk/sms/>

- SDK de Mensagens para Clientes do Chat do IVS: Tutorial do React Native, parte 1:
salas de chat

https://docs.aws.amazon.com/pt_br/ivs/latest/ChatUserGuide/chat-sdk-react-tutorial-chat-rooms.html

- SDK de Mensagens para Clientes do Chat do IVS: Tutorial do React Native, parte 2:
mensagens e eventos

https://docs.aws.amazon.com/pt_br/ivs/latest/ChatUserGuide/chat-sdk-react-tutorial-messages-events.html



Fonte: Flaticon.com